25CSE202 Object Oriented Programming – Mid-Sem II Answer Key

Model answers for the Mid-Sem II question paper. All four programs were compiled and run before being added here.

Unit – III

1 a) Custom exception program (4 marks)

```
import java.util.Scanner;

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) { super(msg); }
}

class Account {
    private double balance;
    Account(double balance) { this.balance = balance; }
    void withdraw(double amt) throws InsufficientBalanceException {
        if (amt > balance)
            throw new InsufficientBalanceException("Insufficient balance.
Available: " + balance);
        balance -= amt;
        System.out.println("Withdrawn: " + amt + ", Remaining: " + balance);
    }
}

public class Q1a {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Account acc = new Account(2000);
        System.out.print("Enter amount to withdraw: ");
        double amt = sc.nextDouble();
        try {
            acc.withdraw(amt);
        } catch (InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
```

```
            System.out.println("Transaction completed.");
        }
    }
}
```

Sample output for 2500: `Error: Insufficient balance. Available: 2000.0` then `Transaction completed.` For 500: `Withdrawn: 500.0, Remaining: 1500.0` then `Transaction completed.`

Marking: custom exception class (1), withdraw() with throw and throws (1), input and Account object (1), try-catch-finally (1).

1 b) throw vs throws, and finally (3 marks)

Point	throw	throws
Purpose	Raises one specific exception object now	Declares exceptions a method may pass to its caller
Where used	Inside a method body	In the method signature
Followed by	An exception instance: `throw new ArithmeticException();`	One or more class names: `throws IOException, SQLException`
Count	One exception at a time	Several, separated by commas

The finally block holds cleanup code that always runs, whether or not an exception occurs or is caught. It is used to close files, connections and other resources.

```
try {
    int r = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Done.");   // always executes
}
```

Marking: difference with two or more points (1.5), finally explanation (1), example (0.5).

2 a) File write and read program (4 marks)

```
import java.io.*;
import java.util.Scanner;

public class Q2a {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of students: ");
        int n = sc.nextInt();

        try (PrintWriter out = new PrintWriter(new
FileWriter("students.txt"))) {
            for (int i = 0; i < n; i++) {
                System.out.print("Enter id, name, marks: ");
                int id = sc.nextInt();
                String name = sc.next();
                double marks = sc.nextDouble();
                out.println(id + "," + name + "," + marks);
            }
        } catch (IOException e) {
            System.out.println("Write error: " + e.getMessage());
        }

        System.out.println("\nID\tName\tMarks");
        try (BufferedReader in = new BufferedReader(new
FileReader("students.txt"))) {
            String line;
            while ((line = in.readLine()) != null) {
                String[] p = line.split(",");
                System.out.println(p[0] + "\t" + p[1] + "\t" + p[2]);
            }
        } catch (IOException e) {
            System.out.println("Read error: " + e.getMessage());
        }
    }
}
```

Sample output for two students: `1 Asha 87.5` and `2 Rahul 92.0`.

Marking: reading n and details (1), writing with PrintWriter/FileWriter (1), reading with BufferedReader (1), try-with-resources and exception handling (1).

2 b) Type wrappers (3 marks)

Primitive types are not objects, but collections, generics and many APIs need objects. Each primitive therefore has a wrapper class in `java.lang`: `Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`, `Character` and `Boolean`. The six numeric wrappers extend the abstract class `Number`.

- **Boxing:** wrapping a primitive in a wrapper object, `Integer iOb = Integer.valueOf(100);`

- **Unboxing:** extracting the primitive from the object, `int i = iOb.intValue();`
- **Autoboxing (JDK 5):** automatic boxing when an object is needed, `Integer iOb = 100;`
- **Auto-unboxing:** automatic extraction when a primitive is needed, `int i = iOb;`

```
Integer iOb = 99;     // autoboxing
++iOb;                // unbox, add 1, rebox -> 100
int j = iOb;          // auto-unboxing
```

Marking: need for wrappers and list of classes (1), four terms defined (1.5), example (0.5).

Unit – IV

3 a) Ways of creating a thread (4 marks)

A thread is a lightweight path of execution that shares the memory of its process. Java offers two ways to create one.

Way 1: extend the Thread class. Override `run()`, create an object and call `start()`.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Running: " + getName());
    }
}
public class Demo1 {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Way 2: implement the Runnable interface (preferred). The class can still extend another class, and the same task can be given to a thread pool. Pass the Runnable object to a `Thread` constructor, or use a lambda.

```
class Task implements Runnable {
    public void run() {
        System.out.println("Running: " + Thread.currentThread().getName());
    }
}
public class Demo2 {
    public static void main(String[] args) {
        new Thread(new Task()).start();
        new Thread(() -> System.out.println("Lambda thread")).start();
    }
}
```

Always call `start()`, not `run()`. `start()` creates a new call stack and the JVM then calls `run()`. Calling `run()` directly just runs it on the current thread.

Marking: two ways with syntax (2), two examples (1.5), start() vs run() (0.5).

3 b) Synchronization program (4 marks)

```
class Counter {
    private int count = 0;
    synchronized void increment() { count++; }
    int getCount() { return count; }
}

public class Q3b {
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();
        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) c.increment();
        };
        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Final count = " + c.getCount());
    }
}
```

Output: `Final count = 2000`. Without `synchronized`, `count++` (read, add, write) can interleave between the two threads. Updates are lost and the result can be less than 2000, which is a race condition. A synchronized method lets only the thread holding the object's lock enter it.

Marking: Counter with synchronized method (1), two threads each looping 1000 times (1), start and join (1), final count displayed (1).

4 a) ArrayList of Employee objects (4 marks)

```
import java.util.*;

class Employee {
    int id; String name; double salary;
    Employee(int id, String name, double salary) {
        this.id = id; this.name = name; this.salary = salary;
    }
    public String toString() { return id + "\t" + name + "\t" + salary; }
}

public class Q4a {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Employee> list = new ArrayList<>();
        System.out.print("Enter number of employees: ");
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            System.out.print("Enter id, name, salary: ");
            list.add(new Employee(sc.nextInt(), sc.next(), sc.nextDouble()));
        }
        System.out.println("\nEmployees:");
        Iterator<Employee> it = list.iterator();
        while (it.hasNext()) System.out.println(it.next());

        System.out.print("\nEnter id to remove: ");
        int rid = sc.nextInt();
        it = list.iterator();
        while (it.hasNext())
            if (it.next().id == rid) it.remove();

        System.out.println("\nRemaining employees:");
        for (Employee e : list) System.out.println(e);
    }
}
```

Sample run with employees 1, 2, 3 and id 2 removed leaves employees 1 and 3. Removal uses `Iterator.remove()`, because calling `list.remove()` inside a for-each loop throws `ConcurrentModificationException`.

Marking: Employee class (1), ArrayList with n objects (1), display with Iterator (1), removal by id and final display (1).

4 b) Collections Framework and ArrayList vs LinkedList (4 marks)

The Collections Framework (`java.util`) gives a standard architecture for storing and processing groups of objects. It uses generics, so element types are checked at compile time.

- `Iterable` is the root, and `Collection` extends it.
- `Collection` has sub-interfaces `List` (ordered, duplicates allowed), `Set` (no duplicates; `SortedSet` and `NavigableSet` keep order) and `Queue` (usually FIFO; `Deque` is double-ended).
- `Map` (key/value pairs, unique keys) is **not** a Collection. Its sub-interfaces are `SortedMap` and `NavigableMap`.
- Implementations: `ArrayList`, `LinkedList`, `HashSet`, `LinkedHashSet`, `TreeSet`, `PriorityQueue`, `ArrayDeque`, `HashMap`, `TreeMap`.

Point	ArrayList	LinkedList
Internal structure	Dynamic (resizable) array	Doubly linked list
Implements	List	List and Deque
Random access `get(i)`	Fast, O(1)	Slow, O(n)
Insert or delete in the middle	Slow, elements shift	Fast once the node is reached, no shifting
Memory	Less per element	More, each node stores two links
Best used for	Frequent reading by index	Frequent insertion and deletion, queue or stack use

Marking: hierarchy with the main interfaces (2), ArrayList vs LinkedList with three or more points (2).